

ChatGPT Handoff — ED Research System (Freeze)

Timestamp: 2025-08-24 12:45 UTC

Scope of this handoff: For future ChatGPT assistants working on the ED Research System
without breaking invariants and in compliance with the contract.

Status snapshot: Phase 1 (Operational Infrastructure) – COMPLETE; Phase 2 (Clinical In
COMPLETE.

Outputs remain ADVICE-ONLY; ML must not flip gates.

Contract (1/3)

Follow this contract exactly. When uncertain ask don't guess.

Invariants (MUST NOT CHANGE/REMOVE)

- WorkflowState unchanged, defined before use; exposes `.feature_dict()`, `.update_state_from_event(...)`, `.apply_event_log(...)`; timers/backlogs/alerts intact.
- TinyCritics uses `WorkflowState.feature_dict()`; cold-start safe (no transform before
- CONFIG bootstrap with defaults; flags `RUN_UI=False`, `RUN_PIPELINE=False`. Paths exist CSV/dir semantics: `EQUIPMENT_STATUS_PATH`, `EQUIPMENT_MOVES_LOG_PATH`, `SOP_REGISTRY_PATH`, `QR_OUTPUT_DIR`, `EVENT_LOG_PATH`.
- Tracker core first-class (`tracker_core.TrackerService` + `tracker_ui.run_ui(...)` or in equivalents). No import-time side effects.
- SOP auto-pull present & offline-safe: `refresh_sop_registry(CONFIG, base_url)`; lazy i returns summary on failure.
- QR: generate + scan-to-update; manual payload fallback must still update moves when absent.
- Overdue/alerts: equipment overdue (by `last_seen`) + lingering patient indicator (by `since_vitals_min`) with adjustable thresholds in UI.
- Notebook metadata has `kernelspec: python3`.

No-hallucinations / verification rules (MANDATORY)

- No claims without evidence: Do not state “implemented,” “wired,” or “working” unless
 - Code cell(s) are added/modified in this notebook or a file write is performed, and
 - A local check/smoke assert for that feature runs in the same response (or in the S and passes.
- Provenance required: When referring to features from older notebooks or sidecars, na and cell/module you copied from or superseded.
- State uncertainty explicitly: If an external dependency, network, or environment cap required, say so and describe fallback behavior. Do not imply it succeeded unless veri
- No virtual resources: Don't reference files/URLs/data that weren't created in this r

Contract (1/3)

present. If missing, create them or mark as TODO—don't claim they exist.

- Exact diffs or full cells: Any modification must include either a line-precise diff or replacement cell/module content.
- No silent renames/moves: If a symbol/path changes, declare the change and update all references in the same patch; otherwise, don't change it.
- Ask before destructive edits: Never delete code/data without an explicit instruction.

Allowed changes

Contract (2/3)

- New features only behind guards: if RUN_PIPELINE: (Phase-2/3) and/or if RUN_UI:.
- Append after core definitions; do not reorder WorkflowState/TinyCritics.
- Optional deps (requests, bs4, Pillow, pyzbar) must be lazy-imported inside guarded c
- You may extend feature_dict() by adding keys (no renames/removals).

Forbidden changes

- No placeholders for WorkflowState. No renames of core classes/functions/paths.
- No top-level execution (no jobs at import; no undefined symbol instantiation).
- No order fragility (no references before definition).
- Do not touch PDFs/data except via tracker/SOP services.

Delivery checklist (MUST PASS)

```
1 Set: CONFIG['RUN_UI']=False; CONFIG['RUN_PIPELINE']=False
2
3 # TinyCritics cold-start
4 s=WorkflowState(role='nurse'); getattr(s,'touch_now',lambda *_:None)(pd.Timestamp.u
5 tc=TinyCritics(); p,b,u=tc.score(s, [{'id':'reassess_vitals','label':'Reassess
vitals'}, {'id':'order_ecg','label':'Order ECG'}])
6 assert len(p)==2 and (0<=p).all() and (p<=1).all()
7
8 # Tracker core basic IO
9 from pathlib import Path
10 from tracker_core import TrackerService, QRService, EquipmentRepository, MovesLogRe
SOPRegistry
11 t=TrackerService.from_config(CONFIG)
12 _=t.equipment_status(); t.log_move('pump-001','A1','B2'); assert
Path(CONFIG['EQUIPMENT_MOVES_LOG_PATH']).exists()
```

Contract (3/3)

```
13 q=QRService(CONFIG['QR_OUTPUT_DIR']).make('pocetest'); assert isinstance(q,str) and
14 sop=SOPRegistry(CONFIG['SOP_REGISTRY_PATH']).read(); assert sop is not None
15 print('SMOKE_OK')
16
17 SOP auto-pull surface present; offline or missing libs returns a non-throwing summa
18 QR scan fallback present: manual payload (e.g., id=pump-001) updates location and a
moves log if decode fails.
19 Overdue/alerts visible in UI with adjustable thresholds (minutes).
20 Notebook metadata contains: {"kernel_spec":{"name":"python3","display_name":"Python
3","language":"python"}}
```

Operational Guidance for ChatGPT

How to reason & act (ChatGPT-specific)

- Always obey the Contract above. If a change is requested, keep it behind guards (if RUN_PIPELINE:).
- Never mutate frozen CONFIG. Use an overlay when tests require flag changes:

```
CFG = {**CONFIG, 'RUN_UI': False, 'RUN_PIPELINE': False}
```
- Provide evidence for claims: include a tiny smoke/print or a file write in the same
- Provenance: if you copy a cell from another notebook, cite the file name and the cell used.
- Avoid virtual resources: Only reference files/paths that exist (or create them explicitly)

Reusable Snippets

Useful snippets (safe to reuse)

1) tracker_core alias shim (if not installed as a module):

```
import sys, types

_names = ['TrackerService', 'QRService', 'EquipmentRepository', 'MovesLogRepository',

if 'tracker_core' not in sys.modules and all(n in globals() for n in _names):
    _m = types.ModuleType('tracker_core')
    for n in _names: setattr(_m, n, globals()[n])
    sys.modules['tracker_core'] = _m
    print('tracker_core alias ready')
```

2) EquipmentRepository.read_robustness_patch (idempotent):

```
import pandas as pd, numpy as np

_EQUIP_COLS = ['equip_id', 'name', 'location', 'status', 'last_seen', 'battery', 'confid

def _equiprepo_read_fixed(self):
    try: df = pd.read_csv(self.status_csv)
    except Exception: return pd.DataFrame(columns=_EQUIP_COLS)
    for col in _EQUIP_COLS:
        if col not in df.columns: df[col] = np.nan
    df['equip_id'] = df['equip_id'].astype(str)
    return df[_EQUIP_COLS]

EquipmentRepository.read = _equiprepo_read_fixed
print('Patched EquipmentRepository.read (robust return)')
```

3) Minimal quickcheck (overlay; optional):

```
def phase1_quickcheck(CONFIG):
    from pathlib import Path; import pandas as pd
```

Reusable Snippets

```
CFG = dict(CONFIG); CFG['RUN_UI']=False; CFG['RUN_PIPELINE']=False

for k in ('QR_OUTPUT_DIR','EVENT_LOG_PATH','SOP_REGISTRY_PATH','EQUIPMENT_STAT
PMENT_MOVES_LOG_PATH'):

    p = Path(CFG[k]); (p.parent if p.suffix else p).mkdir(parents=True, exist_

from tracker_core import TrackerService

t = TrackerService.from_config(CFG)

t.log_move('__qc__','', 'ZZ')

return True
```

Paths & Semantics

Paths & File semantics (from current freeze)

- EQUIPMENT_STATUS_PATH: /kaggle/working/equipment_status.csv (CSV)
- EQUIPMENT_MOVES_LOG_PATH: /kaggle/working/moves_log.csv (CSV)
- SOP_REGISTRY_PATH: /kaggle/working/sop_registry.csv (CSV)
- QR_OUTPUT_DIR: /kaggle/working/qrs (DIR)
- EVENT_LOG_PATH: /kaggle/working/event_log.jsonl (JSONL file)
- MODEL_BUNDLE_PATH: autodetected (e.g., /kaggle/working/ed_phase2_model_thr_patched(1
- Advice outputs: /kaggle/working/advice_gatepos_mlp.csv; optional /kaggle/working/advice_ops_panel.csv

Next Steps (Hold)

Near-term next steps (do not proceed unless requested)

- Phase 2.5–3: build labels and multi-task MLP heads for: clinical complexity, resource availability, 4h deterioration, time-to-dispo, workup completion (advice-only).
- Calibrate each head; report per-task calibration/Brier; integrate as refiners under RUN_UI guard (never veto).
- Add SOP usage telemetry (access counts) under RUN_UI guard.